

SafetyNexus AI: Frontend Action Plan & API Handoff

To: Shubhranshu (Frontend & Integration Lead)

From: Deepak (Backend & AI Lead)

Status: Backend Phase 1 & 2 Complete. Ready for Frontend Integration.

This document contains exactly what you need to know about the backend and exactly what you need to do with it to build the UI views assigned to you in the Work Division plan.

1. Getting the Backend Running on Your Machine

The Info: The entire backend (FastAPI, PostgreSQL, Neo4j, ChromaDB, Redis) is completely dockerized. All synthetic data is automatically seeded on startup.

Your Action Item:

1. Clone the GitHub repository.
2. Create a file named `.env` inside the ``backend/`` folder and add ``GEMINI_API_KEY=your_key_here``.
3. Open your terminal at the root of the project and run: ``docker-compose up --build``
4. Wait for the logs to say ``? Database seeding complete!``. The API is now live at ``http://localhost:8000/api``.

2. Real-Time Data (WebSockets)

The Info: We have a live Socket.io feed pumping out simulated sensor data and alerts every few seconds.

Your Action Item:

- Connect your Next.js app: Use the ``socket.io-client`` library to connect to ``http://localhost:8000``.
- Listen to ``sensor_update`` events: You will receive JSON payloads with sensor IDs, values, and zone coordinates.
 - What to build: Use this data to power the Geospatial Heatmap View (using Leaflet.js). Update the zone colors (green/yellow/red) based on the live readings. Connect this to the Main Dashboard line charts.
- Listen to ``new_alert`` events: You will receive critical risk warnings.
 - What to build: Trigger a Toast Notification immediately. Add the alert to the Alert Management Console list with a red/orange badge depending on the severity.

3. Incident Investigation & RAG

The Info: The backend uses ChromaDB and Google Gemini to allow natural language querying of safety manuals and past incident reports.

- Endpoint: ``POST /api/query``
- Payload: ``{"query": "What are the procedures for hot work near gas sensors?"}``

Your Action Item:

- What to build: Build the Incident Investigation View. Create a chat-like interface or search bar where the Safety Officer can type a question.
- Take the user's input, send it to the ``POST /api/query`` endpoint, and display the AI's response text along

with the source documents it cites.

4. Knowledge Graph & Permit Conflicts

The Info: We are using Neo4j to map relationships between Workers, Zones, Equipment, and Permits. The backend automatically detects if there are conflicting permits (e.g., hot work happening in a zone where a gas leak is detected).

- Endpoint: ``GET /api/knowledge/graph`` (Returns all active relationships)

- Endpoint: ``GET /api/permits/active`` (Returns AI recommendations on whether to approve/deny permits)

Your Action Item:

- What to build: Build the Permit Intelligence View. Fetch the active permits. If ``ai_recommendation`` is "deny" or "approve_with_conditions", highlight that row in RED or YELLOW.

- Render a visual map or list showing which workers are tied to which equipment, based on the Knowledge Graph data.

5. Predictive Analytics (Remaining Useful Life)

The Info: The backend runs Isolation Forest and Linear Regression models to predict when a machine is going to fail based on vibration and temperature trends.

- Endpoint: ``GET /api/predictions/failures``

- Response Data: Returns a list of sensors with their ``rul_days`` (Remaining Useful Life in days) and ``trend`` (e.g., "degrading").

Your Action Item:

- What to build: On the Main Dashboard, create an "Equipment Health" widget. Fetch this endpoint and display a warning banner for any equipment where ``rul_days`` is less than 7 days. Show the trend data in a simple progress bar (e.g., 100% health dropping to 10%).

End of Document. Good luck with the UI!